CS6160 Complexity

Topic: Gödel's Theorem, Complexity Classes, Reductions, Completeness — Date: Aug 31, 2026

Lecturer: Wei-Kai Lin — Scriber: Deebakkarthi Chinnasame Rani

---

1 Gödel's Incompleteness Theorem [Göd31]

As the calender flipped to the 20^{th} century, David Hilbert put forth a set of questions [Hil02] that would steer the course of mathematical research for the ensuing 100 years. His second question titled *The compatibility of the arithmetical axioms* asks the following:

> *To prove that they [axioms] are not contradictory, that is, that a finite number of logical steps based upon them can never lead to contradictory result*

He followed this up by formulating *Hilbert's Program* in the 1920s[Zac23]. It asks for *Formalization, Completeness, Consistency, Conservation, Decidability.* Stated more plainly Hilbert's question asks "Is there a system of axioms that can prove **all** true mathematical statements (completeness) while **not** proving any false statements(soundness)?". These properties can be trivially satisfied individually - A system that proves everything is complete and a system that doesn't prove anything is sound.

In 1931, Kurt Gödel arrived at a gloomy conclusion - Any sound system of axioms cannot be complete.

Theorem 1. *Gödel's Theorem [AB09]*

For every sound system $\mathcal{S}$ of axioms and rules of inference, there exists true number theoretic statements that cannot be proved in $\mathcal{S}$

We need to formally define what these terms mean. Since, this is a complexity class, let us formally define them in terms of Turing machines.

Definition 2. Proof System

Let $\mathbb{T} \subseteq \{0,1\}^*$ be the set of all *true* statements.

TM V is a proof system for $\mathbb{T}$ if it has all the following properties:

Effectiveness

$$V(x,w) = 0 \text{ or } 1, \forall (x,w) \in \{0,1\}^*$$

TM V halts and outputs a bit for all inputs and witnesses. This means that TM V **always halts**.

Soundness

$$V(x,w) = 0, \forall x \notin \mathbb{T}, \forall w \in \{0,1\}$$

A false statement cannot be proved no matter the witness.

Complete

$$\exists w \in \{0,1\}^* \text{ s.t } V(x,w) = 1, \forall x \in \mathbb{T}$$

There is a witness for all the true statements such that V proves it.

Gödel's theorem is a classic case of the *Inconsistent triad*. A trope prevalent in Computer Science, where no system can possess the triad simultaneously.

Though there is a universal quantifier in front of Gödel's theorem, we only care about *interesting* $\mathbb{T}$s. It is trivially easy to define useless systems for which the theorem holds. What makes a $\mathbb{T}$ *interesting*? Without explicitly proving it, we say that any *interesting* mathematical system should be able to express a Turing machine. Therefore if we prove that no TM can satisfy the triad, we've proven that no useful mathematical system can satisfy the triad. We prove Theorem 1 using a chronologically later proof by Alan Turing.

Theorem 3. *[Tur37]*

Any proof system that can express a Turing Machine ***cannot*** *be effective, sound and complete simultaneously.*

This is a corollary of Turing's solution to Hilbert's *Entscheidungsproblem* (decision) problem.

Proof. [Bar21]

Let $\mathbb{H} = \{< \alpha, x > | M_\alpha(x) \text{ doesn't halt}\}$ be our $\mathbb{T}$ i.e our $\mathbb{T}$ is the set of all non-terminating TMs. Given a string $\langle \alpha, x \rangle$ if one could decide if it halts or not then we would have proved all of $\mathbb{H}$. As you might have guessed by seeing the word "halt", the proof involves reduction to HALT.

We claim that $\nexists$ TM V that is effective, sound and complete w.r.t $\mathbb{H}$

Assume for the sake of contradiction that $\exists V^*$ that is effective, sound and complete w.r.t $\mathbb{H}$

i.e $V^*(< \alpha, x >, w)$ can tell us if $M_\alpha(x)$ halts or not.

We construct the following TM M as shown in Figure 1 to solve the HALT problem

$M(\alpha, x)$: Run the following in "parallel"[1]

- Execute the next step of $M_\alpha(x)$, if it halts output 1
- Pick the next $w \in \{\text{“”}, 0, 1, 00, 01, \ldots\}$ and run $V^*(x, w)$. If it outputs 1, output 0

We find that either $M_\alpha(x)$ halts or we find a valid w that makes V^* accept, thereby solving HALT.

But we know that HALT is not computable therefore no such V^* cannot exist.

If no such TM V^* can exist then no such *useful* proof system can exist, thereby proving Theorem 1. □

1.1 Relation between Uncomputability and Incompleteness

The above proof may not be the most intuitive. But it is much simpler than Gödel's original proof as we can use a formal language(TM) that we are familiar with. We are using the fact that Turing machines are a universal model of computation and therefore can perform any mathematical

[1] Don't mistake this for non-determinism. M operates very similarly to how a single core CPU can multi-thread and give the illusion of concurrency.

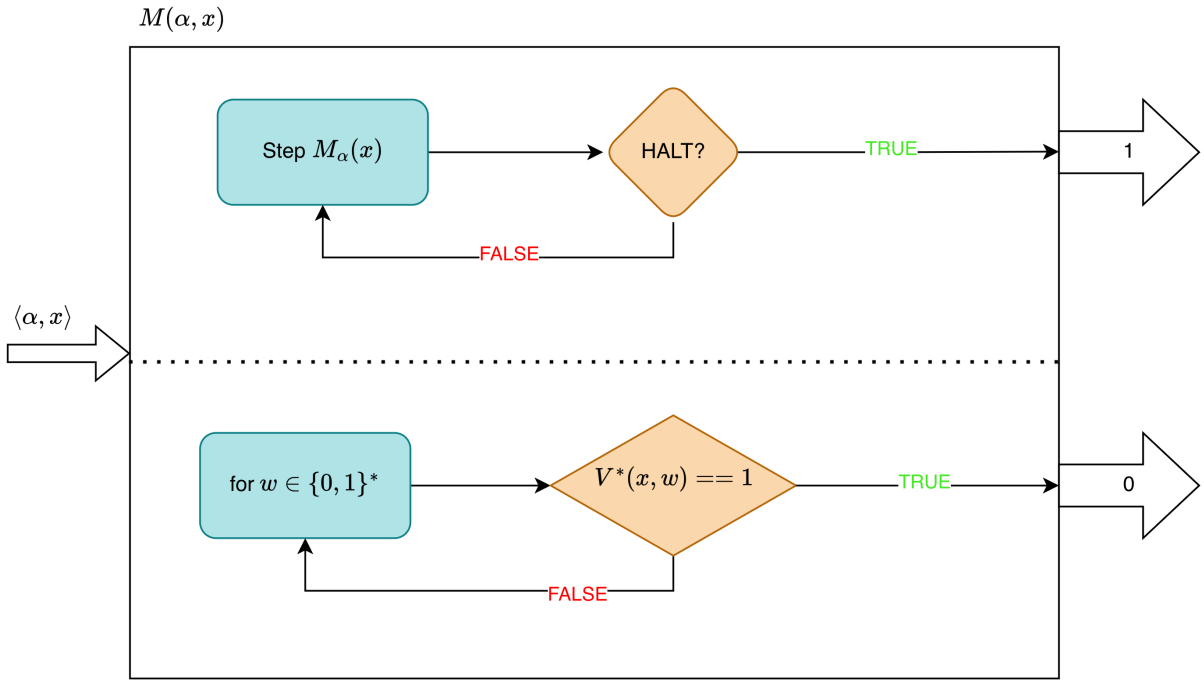

Figure 1: Visual Representation of the Reduction

operation. We flip this relation on its head, saying that any sufficiently complex mathematical system should be able to describe a Turing machine. Therefore *all useful* mathematical systems necessarily have at least one true statement that they cannot prove - HALT.

Turing's proof may even be slightly stronger than Gödel's as proving something as inherently *uncomputable* is harder than proving that a system is merely *incomplete*.

2 Complexity Classes

Definition 4. Class **DTIME**

The complexity class **DTIME**, parameterized by $T(n)$, where $T : \mathbb{N} \to \mathbb{N}$, is defined as

$$\{F : \{0,1\}^* \to \{0,1\} | \exists \text{ TM } M(x) \text{ that runs in } c \cdot T(n), \text{ for some } c > 0 \text{ and computes } F(x)\}$$

DTIME is the set of all functions that can be computed in $O(T(n))$ time by some TM M. Notice how $O(\cdot)$ is baked into the definition, making future asymptotic analysis much simpler.

Definition 5. Class **P**

$$\mathbf{P} = \bigcup_{c \geq 1} \mathbf{DTIME}(n^c)$$

Class **P** is the set of all functions that can be computed in *polynomial* time by some TM M.

The **D** in **DTIME** refers to *Deterministic*. This is because all the Turing machines referenced above are deterministic. The reason for this distinction will be made clear in the next section.

Class **P** is what we colloquially refer to as *easy* problems. Historically, if a problem was proven to be in **P**, soon it was also proven to be in **DTIME**(n^2).[2]

2.1 Non-deterministic complexity classes

Before we introduce the non-deterministic complexity classes **NTIME** and **NP**, we need to define additional TMs.

Definition 6. Verifier

A verifier is a TM that takes a second input string called a *witness* or a *certificate*.

For a language L,

$$x \in L \iff \exists\ w \text{ s.t. } V(x, w) = 1$$

For every valid x there is some valid witness w such that V accepts.

As an example, consider a verifier for 3SAT. It takes some CNF ψ along with a string that satisfies ψ. It is pretty easy to see how this verifier can decide whether ψ is in 3SAT.

Notice how we are not concerned with how w came about. We just assume that some w will be provided. In most problems w roughly corresponds to the *correct* answer of the problem and the verifier merely checks it.

Definition 7. Non-Deterministic Turing Machine (NDTM)

A NDTM is just like a TM except it has two transitions functions δ_0, δ_1 and has a special state called q_{accept}. At each state, we envision the NDTM arbitrarily picking one of its transitions functions to apply. We say that the NDTM accepts if there is a sequence of choice of transition functions such that q_{accept} is reached. If no such sequence exist, we say that the NDTM rejects.

Intuitively we think of the NDTM as *forking* during every state, executing δ_0 in one thread and δ_1 in the other. This analogy can be extended to much more than 2 parallel threads by just using some kind of binary encoding.

2.1.1 Equivalence of NDTM and Verifier

NDTM is equivalent to a verifier.

i.e $\forall$ NDTM M, $\exists$ a Verifier V such that $M(x) = 1 \iff \exists w, V(x, w) = 1$

We construct the verifier as follows: For every $x \in L$ recognized by M, we can construct a witness w that represents the choice between δ_0 and δ_1 at each state. A verifier which is the exact replica of M but without any of the non-determinism can just use the witness w to arrive at the same result.

With this relatively simple conversion, we can think of NDTM as just a TM with an extra input. This greatly helps in proofs as non-determinism may be really hard to work with.

[2] This is the reason why *efficient* in DSA usually means anything sub-quadratic.

The concept of this magic witness tape is often also used to model *Apriori* knowledge in cryptographic schemes.

Definition 8. NTIME

Similar to **DTIME**, we define **NTIME**, parameterized by $T(n),\ T : \mathbb{N} \to \mathbb{N}$ as

$$\left\{ F : \{0,1\}^* \to \{0,1\} \middle| \begin{array}{l} \exists \text{ TM } V \text{ s.t.} \\ \forall x, F(x) = 1 \iff \exists w \in \{0,1\}^*, |w| \leq T(|x|) \text{ s.t. } V(x,w) = 1 \text{ and} \\ V \text{ runs in } T(|x|) \end{array} \right\}$$

NTIME$(T(n))$ is the set of all functions that can be *verified* by some TM V in time $T(n)$

The **N** in **NTIME** stands for *Non-Deterministic*. In fact, **NTIME** was historically defined using NDTM. A equivalent definition using NDTM, given in [AB09], is as follows

$$\{F : \{0,1\}^* \to \{0,1\} | \exists \text{ NDTM } M \text{ that runs in } O(T(n)) \text{ and computes } F(x)\}$$

This mirrors definition 4 with TM swapped out for NDTM.

Definition 9. NP

$$\mathbf{NP} = \bigcup_{c \geq 1} \mathbf{NTIME}(n^c)$$

The class **NP** is the set of all functions that can be *verified* by a polynomial time TM given a polynomial size *witness*.

Equivalently, **NP** is the set of all functions that can be *computed* by a polynomial time NDTM.

3 Reductions

We move on to *Reductions*, a fundamental concept in complexity. We've already seen it being used multiple times in this lecture. It is one of the main techniques we use to prove theorems.

Definition 10. Karp Reduction

If we have two functions $A, B : \{0,1\}^* \to \{0,1\}$, we say that A *karp-reduces* to B if $\exists$ a polytime TM R such that $A(x) = B(R(x)), \forall x \in \{0,1\}^*$[3].

We denote this by $A \leq_p B$

[3]This is an abuse of notation. R is not a function and therefore cannot be composed. We merely use this notation to suggest that the value computed by R is being fed into B

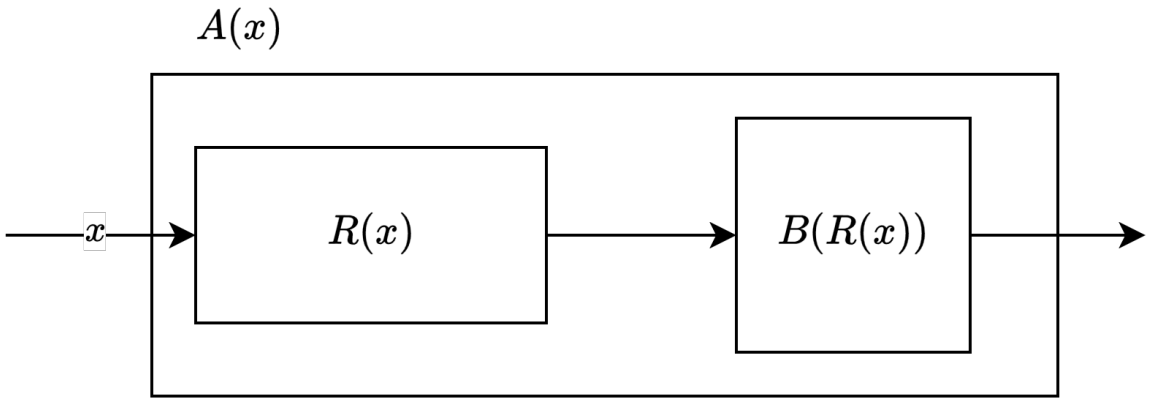

Figure 2: Visual representation of Karp reduction

An easy way to remember which goes where is shown in Figure 2. Reading from left to right, absorb functions in the direction of the inequality arrow.

Definition 11. Cook Reduction

For two functions $A, B : \{0,1\}^* \to \{0,1\}$, We say that A *cook-reduces* or *reduces* to B if $\exists$ TM R with oracle access to B such that

$A(x) = R^B(x)$

This roughly means that R can use B polynomially many times, whenever it wants.

We denote this as $A \leq_T^P B$

Cook reductions are more general and are used more often than Karp reductions. Karp reductions are mostly used when dealing with **NP**-complete reductions.

4 Completeness

As a primer for future lectures, let us define some important reducibility-induced complexity classes, namely **NP**-hard and **NP**-complete. These combine the definitions of reductions and complexity classes we saw above to define new ones.

Definition 12. **NP**-hard

A function $F : \{0,1\}^* \to \{0,1\}$ is **NP**-hard if $\forall G \in \mathbf{NP}, G \leq_p F$

As the name suggests, if a problem is **NP**-hard, then it is just as hard or *if not harder* than any problem in **NP**.

Knowing a solution to solve F will solve all problems in **NP**.

Pay attention to the *if not harder* part. F doesn't necessarily have to be in **NP**.

Definition 13. **NP**-complete

A function $F : \{0,1\}^* \to \{0,1\}$ is **NP**-complete if it is **NP**-hard and if $F \in \mathbf{NP}$

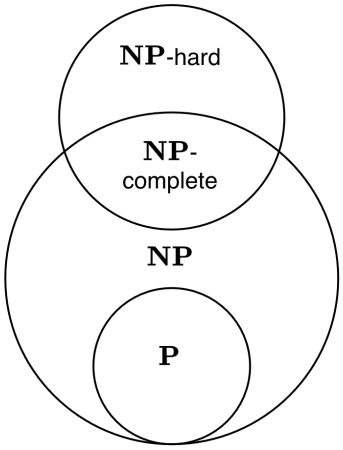

Figure 3: Hierarchy of Complexity classes

The venn diagram in Figure 3, shows the hierarchy of complexity classes assuming $\mathbf{P} \neq \mathbf{NP}$

References

[AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[Bar21] Boaz Barak. *Introduction to Theoretical Computer Science*. 2021. Online textbook.

[Göd31] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatsh. Math. Phys.*, 38(1):173–198, 1931.

[Hil02] David Hilbert. Mathematical problems. *Bull. New Ser. Am. Math. Soc.*, 8(10):437–479, 1902.

[Tur37] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1937.

[Zac23] Richard Zach. Hilbert's Program. In Edward N. Zalta and Uri Nodelman, editors, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Spring 2023 edition, 2023.